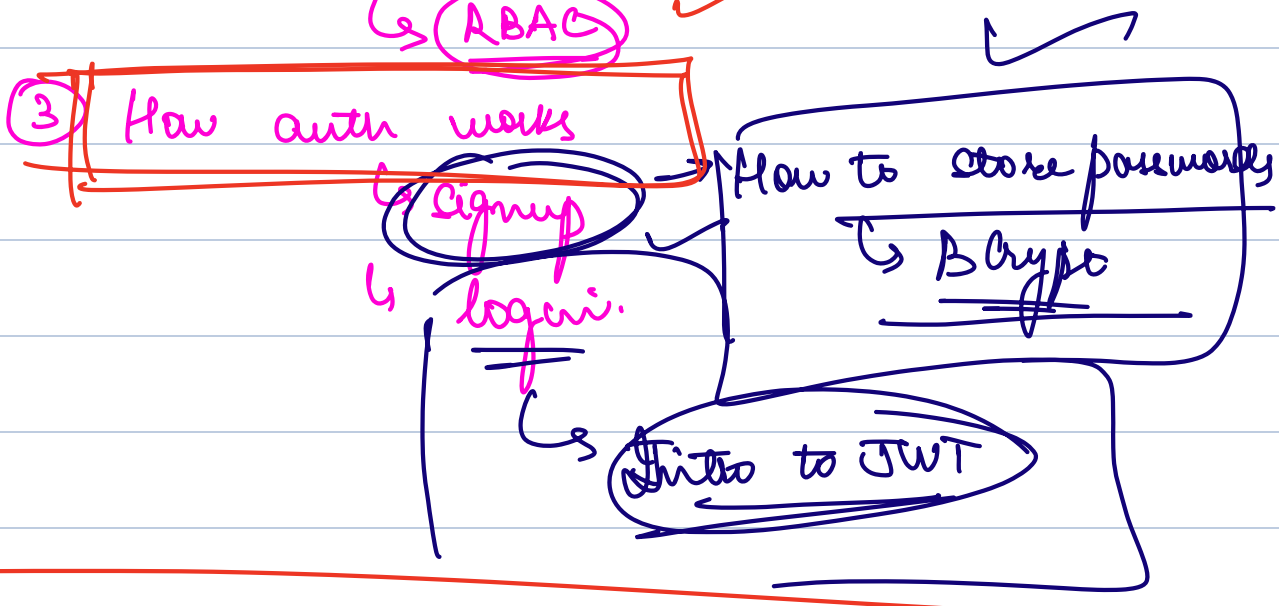
Authentication

1. Authentication ?
2. Authorization ?
   - ABAC
3. How auth works
   - signup
   - login.



Authentication

Public

1. Scaler.com/events
   - anyone can visit this page
   - Scaler may not even know their name.
2. Register for event
   - anyone can register.
   - BUT they have to give their info
   - KYC
   - unless Scaler knows who you are, you can't register.

Authentication (telling who you are)

(3) Scaler.com/admin

⇒ not everyone can visit

→ tell who you are

Authorization

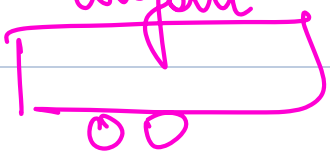
Authentication

+

Having Permissions

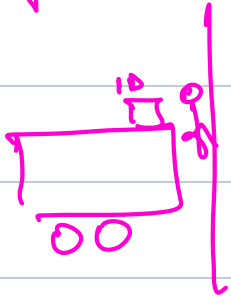
{ you should have necessary permissions }

Visit



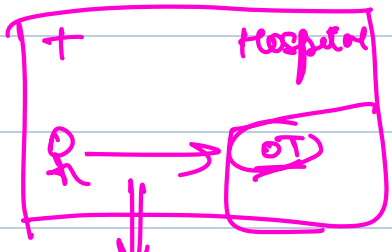
No Auth

Entry to hospital



Auth

visit OT



permissions

Authority

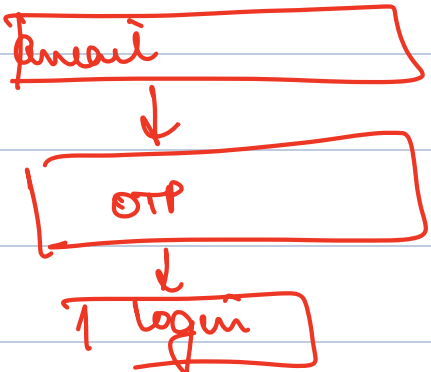
Auth + Auth

How do we authenticate

→ email, password

→ mobile#, OTP

→ Google / FB / Github



How do we authorize?

RBAC
- Role
- Based
- Access
- Control

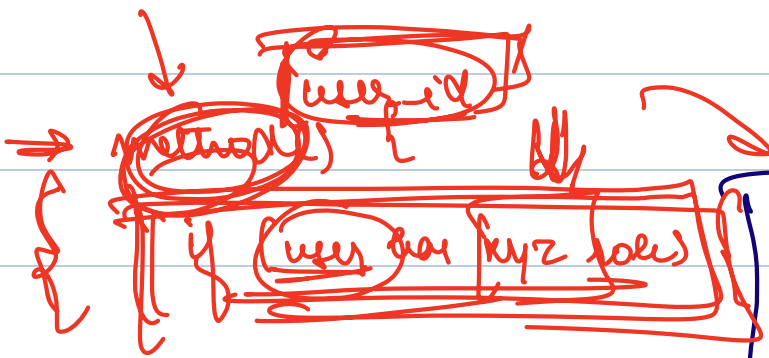
users

id	email	pass

roles

id	name
1	ADMIN
2	MENTOR
3	MENTEE
4	INSTRUCTOR

⇒ put RBAC at diff API endpoints



user_roles

user_id	role_id
1	1
1	2
1	3
2	1
2	3
3	2
4	1
4	2

} else {

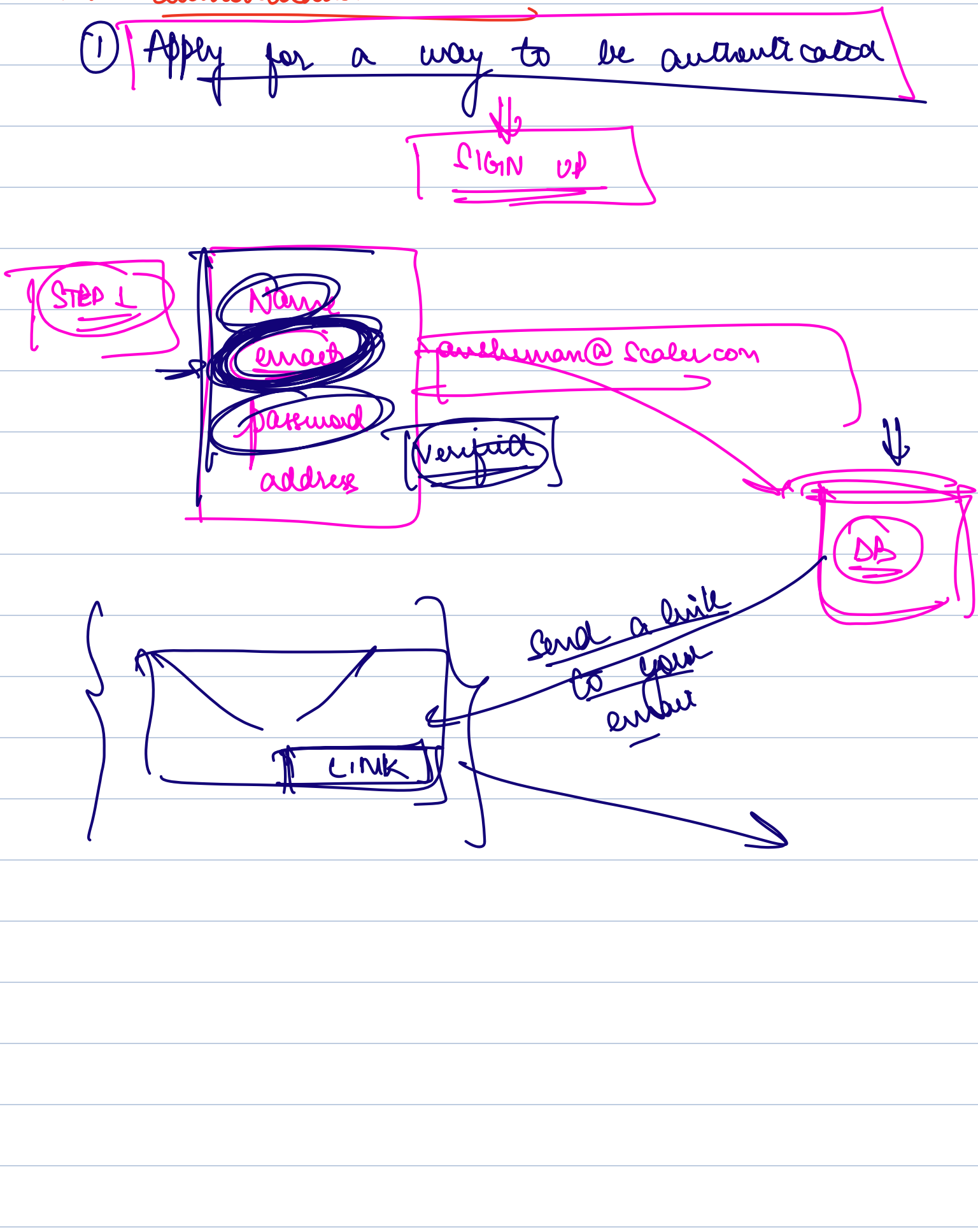
throw Exception()

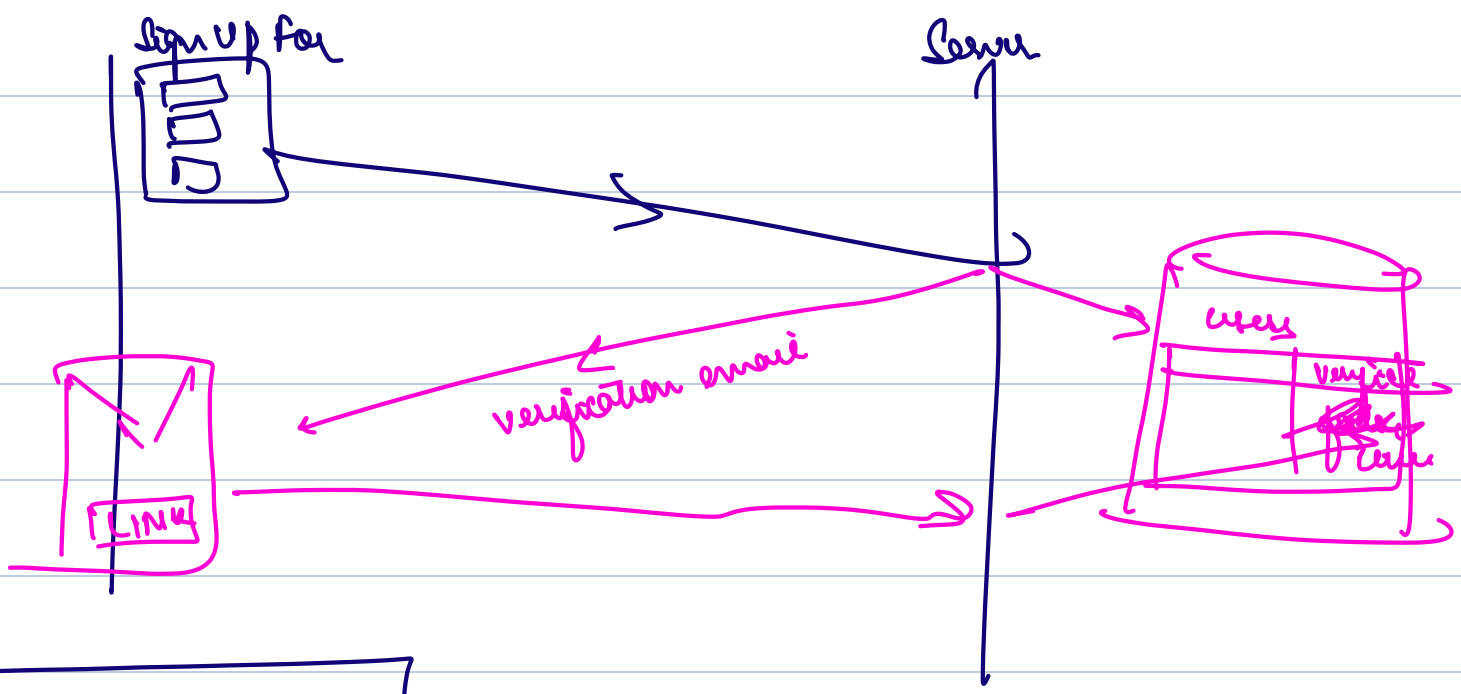
}

admin [1, 3]

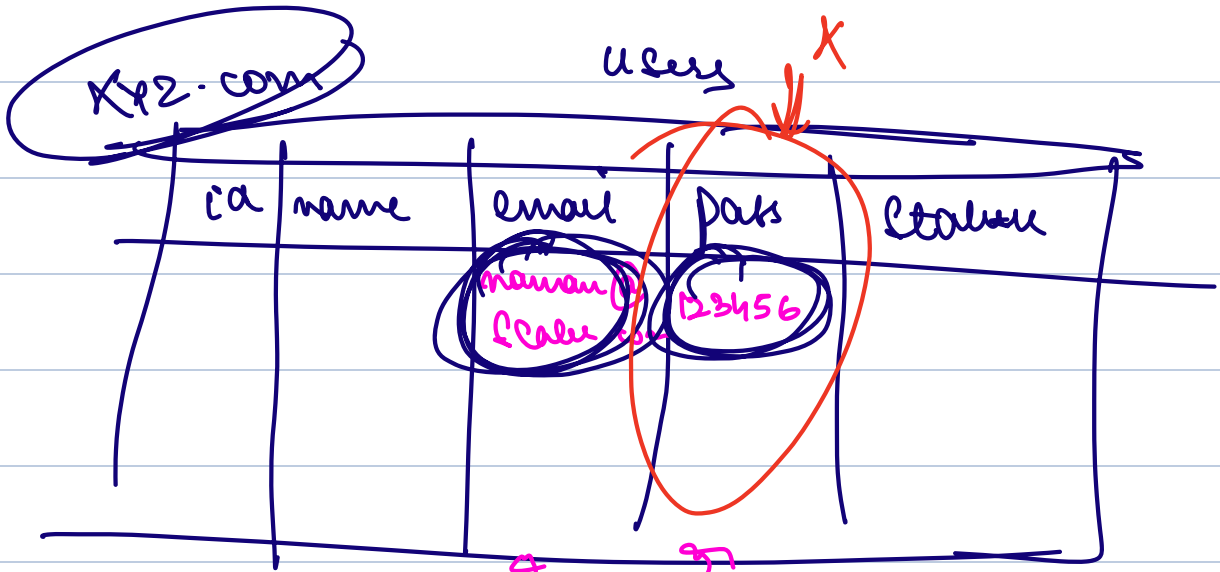
How authentication works

1. Apply for a way to be authenticated





Email + Pass



⭐ Storing passwords as plaintexts is a terrible idea!

⇒ DB leak : people have same pass on multiple places.

⇒ your.env can leak

Hashing

123456 → hash(123456) → abcdef

hash(123456) → abcdef

hash(123456) → xyzabc

login [email, pass]

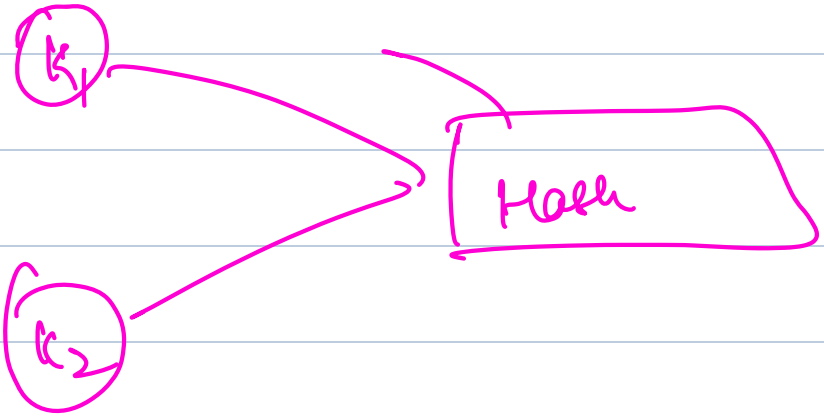
email → hash(pass)

Yes

⇒ hashing same key always gives same value.

hash(naman) → xyz123

getting original pw from hashed not possible.



login(email, pass)

in DB y I have email, hash(pass)

BAD Idea

⇒ many people have same common pass.

"query"

"password"

Case

DB gets leaked

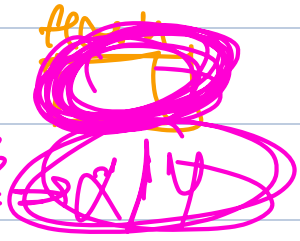
Hacker creates multiple accounts with common password.

a@ scam

pass ⇒

query

i love you.



get to know more!

Hash + Salting ⇒ some random value

raman@scaler.com → password

hash(password + curTime) → XBY def

sakshar → password

hash(password + curTime) → abc 12 abc

different

login(raman@scaler.com, password)

hash(password, curTime)

Special libraries (Algorithms)

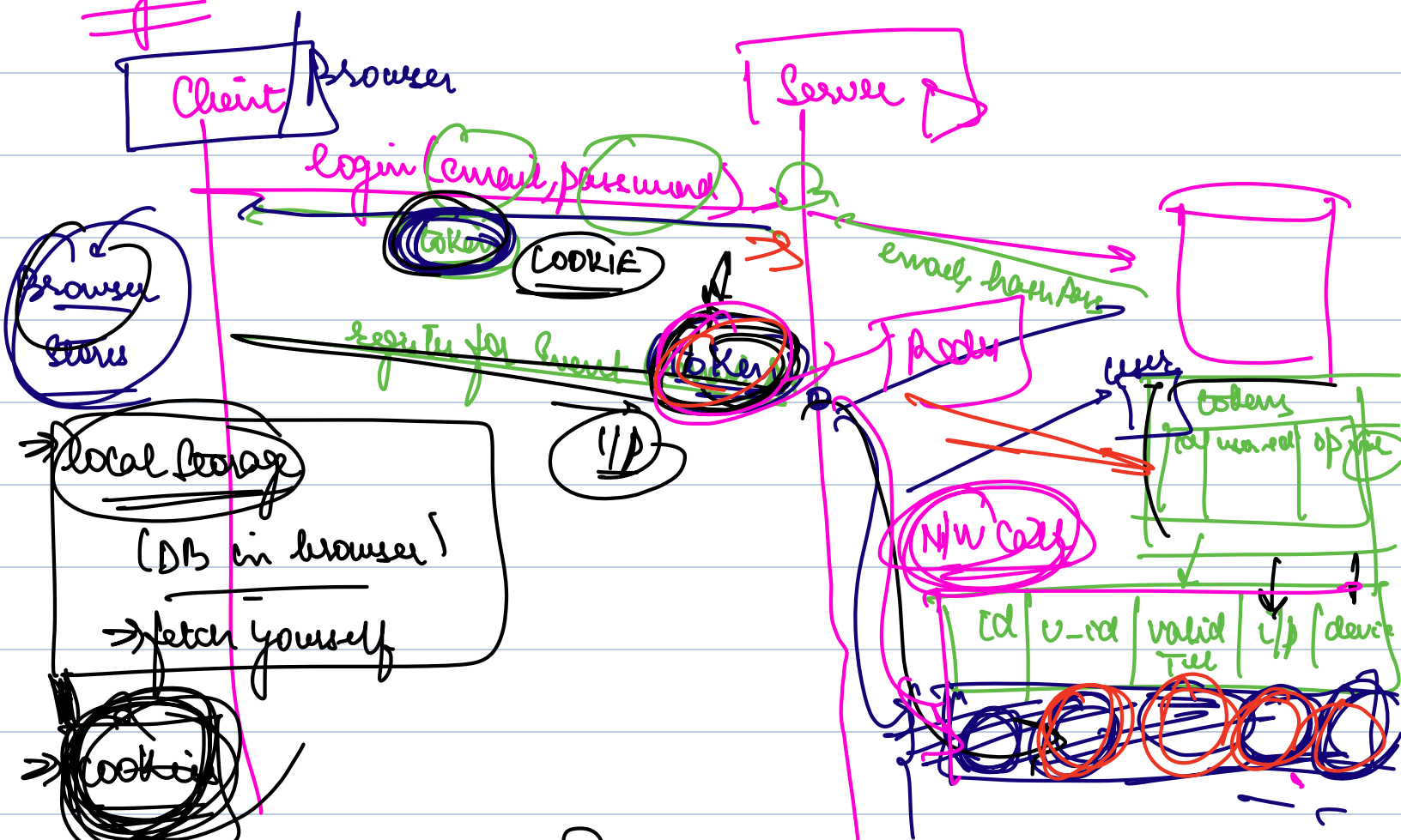
BCrypt Password Encoding Algo

1. encode(password) ⇒ $2a$12$...
   encode(password) ⇒ $2a$12$...

2. verify($2a$12$..., password) ⇒ true

   verify($2a$12$..., password) ⇒ true

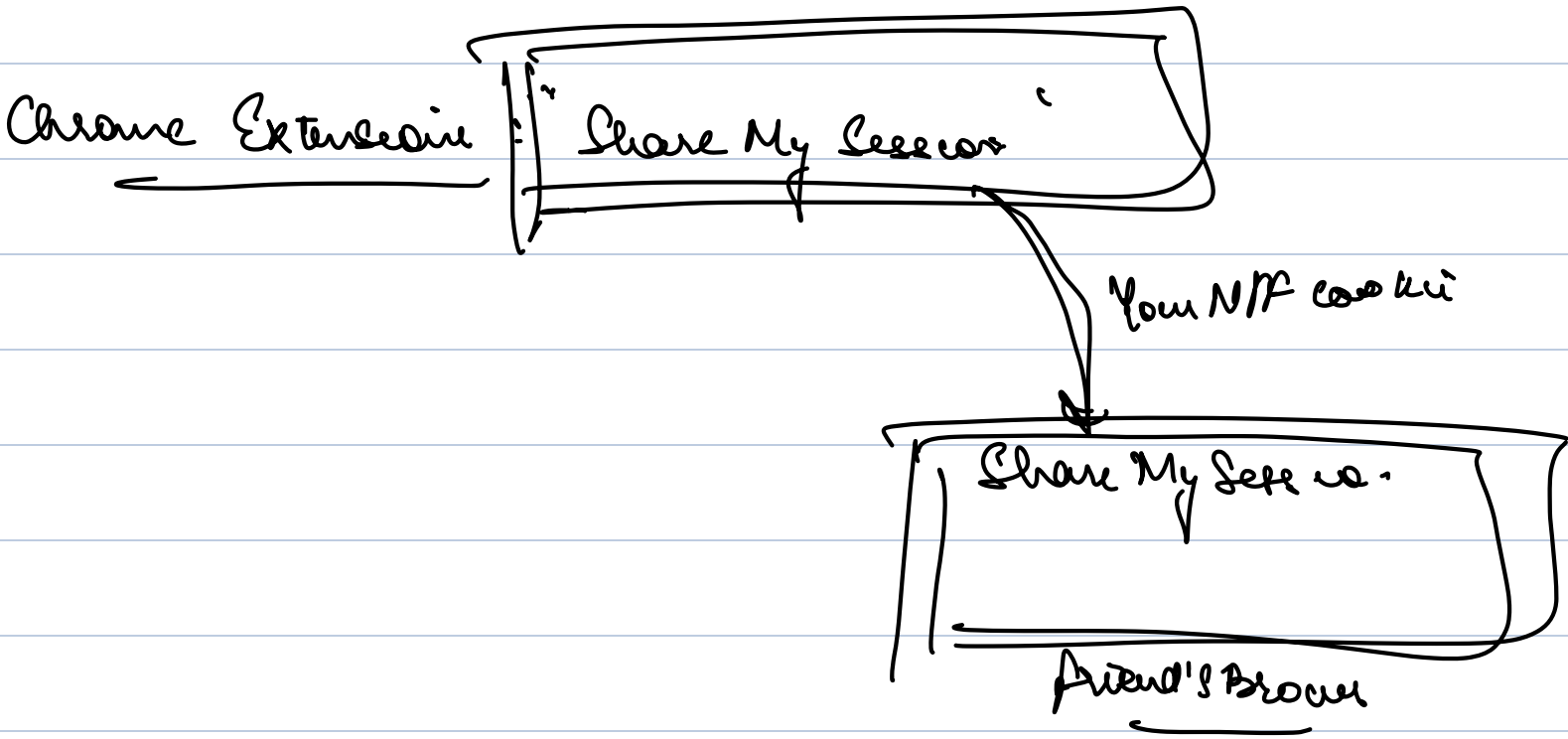
Login



→ every future req will automatically also get all cookies attached.

» Logout from all devices

→ Login Limit



How can we make token validation fast

→ Cache

» Token validated itself

50k serving

1h

1h ~ 50s

→ 50GB

JWTs

JSON Web Tokens

Build a token in a way that within itself token had every info to validate itself.

- ⇒ what if ip, device, valid till, u_id
- ⇒ were stored in token itself

Sat ⇒ JWT & OAuth